

KnowDemo: Knowledge-Guided Robot Demonstration Generation from Human Videos

Zhiyuan Gao¹, Yanxiang Zhan¹, Mohammad Khoshnazar¹, Jeroen Schäfer¹, and Michael Beetz^{1,2}

¹University of Bremen

²Robotics Institute Germany (RIG)

Abstract—Learning robot manipulation policies typically requires substantial demonstration data, which are costly to collect on real robots. Recent methods generate robot demonstrations from human videos by adapting recovered motion and validating the resulting trajectories in simulation. However, methods centered on motion-reference adaptation can limit behavioral diversity by retaining the demonstrated contact strategies and subtask orders, while insufficient understanding of task requirements and scene relations can reduce demonstration generation efficiency by generating invalid candidates. To address these limitations, we propose KNOWDEMO, a framework that uses structured manipulation knowledge from human videos to generate diverse robot demonstrations for a target workspace. To distinguish task requirements from demonstration-specific choices, we develop a knowledge extraction and reasoning module based on a vision-language model (VLM) that associates object and action descriptions with inferred task conditions, demonstration references, and permissible execution variations. To translate this knowledge into executable demonstrations, we resolve the descriptions against target-scene entities and geometry to guide candidate generation and screening before motion planning and simulation. The resulting demonstrations exhibit multimodal behavior through alternative contact strategies and valid subtask orders, with structured execution labels. Experiments demonstrate additional verified execution modes beyond a reference-only configuration and improved candidate planning success through task-guided grasp sampling. To validate the generated data for policy learning, we fine-tune the pretrained $\pi_{0.5}$ model on simulation data, achieving sim-to-real transfer across three tasks. Project page: <https://zhiyuan-gao.github.io/knowdemo/>.

I. INTRODUCTION

Robot demonstration datasets support learning manipulation policies across tasks and environments [1], [2]. However, real-robot demonstration collection remains costly [3], [4]. Human videos offer accessible task knowledge and motion evidence for generating robot demonstrations [5], [6], [7]. Turning these videos into robot training data requires generating observations and actions for the target robot and its workspace. Video-to-robot approaches extract task plans or recover motion to guide execution [8], [9], [10]. Demonstration-generation methods expand datasets by adapting reference motions to new object poses and geometries [3], [11]. Task and scene knowledge further guide execution by grounding abstract action descriptions in the current environment [12] and incorporating semantic and

geometric constraints into motion planning [11], [13], [14]. Simulation subsequently tests the resulting trajectories for physical feasibility [10], [15]. These efforts extend demonstration generation across scene and object variations.

However, adapting recovered object motions to new scene configurations can retain the demonstrated contact strategy and subtask order. The resulting data may cover different object configurations while capturing only a limited range of ways to complete the task. A single task can admit multimodal behavior [16], [17], [18]. For example, the person in the video may close a drawer by holding its handle, whereas a robot could achieve the same goal by pushing the front panel. Generating such alternative behaviors requires distinguishing essential task conditions from demonstration-specific choices.

Candidate generation must also account for subsequent subtasks and target-scene geometry. A grasp that permits object pickup may obstruct later placement or gripper withdrawal. Ignoring these requirements can lead to failed planning attempts. Grounding task knowledge in the target scene allows these requirements to guide candidate generation before motion planning and simulation.

To address these challenges, we propose KNOWDEMO, a framework that uses structured manipulation knowledge from human videos to generate diverse robot demonstrations for a target workspace (Fig. 1). To distinguish task requirements from demonstration-specific choices, our knowledge extraction and reasoning module based on a vision-language model (VLM) associates structured object and action descriptions with inferred task conditions, observed contacts and motions, and permissible execution variations. To translate this knowledge into executable demonstrations, we resolve the descriptions against target-scene entities and geometry, using the grounded task conditions to guide candidate sampling and screening before motion planning and simulation. Each accepted demonstration retains the task instruction inferred from the video and structured execution labels, allowing data to be organized and selected by execution mode. The resulting simulation demonstrations support policy fine-tuning for deployment in the target workspace. We quantify behavioral diversity by comparing the number of verified execution modes with a reference-only configuration, supported by examples of alternative contact strategies and subtask orders from the same human video. A shared-backend ablation measures the effect of task-guided grasp sampling on candidate

Email addresses, in author order: gao@uni-bremen.de, yanxiang@uni-bremen.de, khoshnam@uni-bremen.de, jeroen.schaefer@uni-bremen.de, and beetz@informatik.uni-bremen.de.

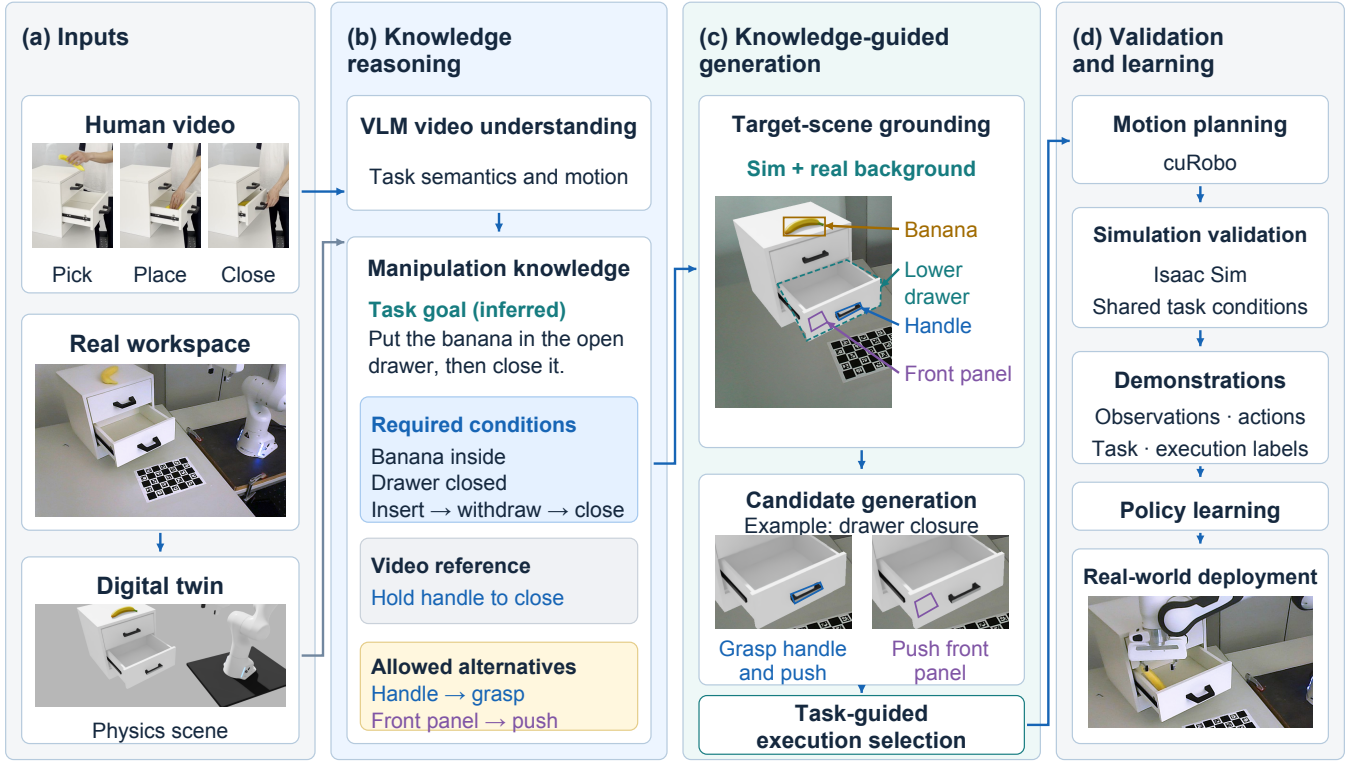


Fig. 1. Overview of KNOWDEMO. (a) Pick, place, and close frames from a human demonstration, the target workspace, and its digital twin. (b) A drawer-closure example links an inferred task goal to required conditions, the video reference, and allowed alternatives. (c) Colors link alternatives to grounded regions and candidate contacts. Rendered robots and objects are composited onto the real background to form policy observations. (d) Planning and simulation validate candidates, yielding demonstrations with shared task instructions and execution labels for $\pi_{0.5}$ fine-tuning and real-world deployment, illustrated by banana placement during drawer storage.

planning success. To validate the generated data for robot learning, we fine-tune the pretrained $\pi_{0.5}$ vision-language-action (VLA) model [19] on the generated simulation demonstrations and demonstrate sim-to-real transfer.

In summary, our main contributions are:

- To generate robot training data from human videos, we introduce KNOWDEMO, a framework that uses structured manipulation knowledge to generate diverse demonstrations for a target workspace.
- To distinguish task requirements from demonstration-specific choices, we develop a knowledge extraction and reasoning method that associates task conditions, demonstration references, and permissible variations with structured object and action descriptions. To translate this knowledge into executable demonstrations, we ground object and action descriptions in target-scene entities and geometry to guide candidate sampling and screening before motion planning and simulation, retaining structured execution labels with the generated demonstrations.
- To evaluate the framework, we quantify gains in verified execution modes over a reference-only configuration, measure the improvement in candidate planning success, and validate sim-to-real transfer using policies fine-tuned on the generated data.

II. RELATED WORK

A. Manipulation Knowledge from Human Videos

Human videos support robot manipulation through two common approaches: training policies that map robot observations to actions, and extracting explicit knowledge representations of goals, object relations, and motion references for planning. In the first approach, MimicPlay learns latent plans from human play to guide a policy trained on robot demonstrations [20], while Vid2Robot conditions actions on demonstration videos and current observations [5].

Explicit representations expose task structure and motion for reasoning and adaptation. SeeDo converts videos into action plans through vision-language reasoning [8]; ORION represents object states, contacts, and motion with Open-World Object Graphs [21]; DITTO transforms object-relative motion for the current scene [9]. DemoDiffusion combines motion references with learned action priors, refining re-targeted human motion using a pretrained diffusion policy [22]. Our structured knowledge separates task requirements, demonstration references, and permissible variations to guide target-scene demonstration generation.

B. Knowledge-Guided Robot Demonstration Generation

Task and scene knowledge guides contact selection and motion parameters. CRAM represents objects, locations, and actions through designators; perception and reasoning

resolve their unspecified properties to contextualize generalized plans [12]. KnowRob combines symbolic knowledge, perception, geometry, and simulation for manipulation reasoning [23]. Spatial constraints and value maps provide geometric guidance for planning [24], [25], [26]. Task-oriented grasping connects contact selection to the intended operation: OVAL-Grasp uses task-relevant part heatmaps to score and filter grasp candidates [27], and DemoFunGrasp conditions grasps on affordance regions and grasping styles [28].

Demonstration generators make these decisions across scene configurations. MimicGen and DemoGen adapt references to new object poses [3], [4]; SkillMimicGen combines skill-level generation with planning [13]. CP-Gen preserves keypoint-trajectory constraints across geometries [11], HybridGen uses VLM segment classification and constraint-guided replanning [14], and AffordGen transfers affordance keypoints across meshes [29]. Using human videos, DexImit, PSI, and Real2Gen combine motion synthesis with simulation [10], [15], [30]. Pegasus combines task, affordance, and constraint graphs with physical verification [31].

We adopt CRAM’s designators and resolution mechanism to link video-derived task conditions, references, and permissible variations to action descriptions. Target-scene resolution specifies alternative contact regions, interaction modes, grasp strategies, and valid subtask orders. Generated demonstrations retain these choices as execution labels under the shared task instruction.

C. Scene Reconstruction for Robot Simulation

Video2Policy, Human2Sim2Robot, and X-Sim reconstruct interactive environments for learning from human videos [7], [32], [33]. Articulated-object reconstruction supplies geometry and joints [34]. For visual matching, SIMPLER composites simulated assets onto real backgrounds [35], while SplatSim and RoboSimGS use Gaussian splatting [36], [37]. Our target-workspace digital twin supplies entity references and geometry for knowledge grounding and demonstration generation across tasks.

III. METHOD

KNOWDEMO comprises four components: digital-twin construction (Section III-A), manipulation knowledge extraction and reasoning (Section III-B), knowledge-guided execution generation (Section III-C), and simulation validation and policy learning (Section III-D). Figure 1 shows the framework.

A. Problem Setup and Digital-Twin Construction

Given a human video V , calibrated target-workspace RGB observations Y_R (optionally with depth), and a robot model M_R , we generate an observation–action dataset D_R . Demonstrations satisfy the inferred task requirements while allowing alternatives to the observed contacts and subtask order.

We construct a digital twin $\mathcal{E}_R$ of the target workspace in Isaac Sim, using an official Franka model, reconstructed task objects, and an aligned collision table. The table provides physical support and collision geometry but is excluded from

the rendered RGB foreground. Policy observations combine robot and object renderings with a fixed-camera workspace image after masking and inpainting movable entities [35]. SAM3 [38] segments images, SAM3D Objects [39] reconstructs rigid objects, and URDF-Anything+ [40] reconstructs articulated task objects, including drawer cabinets and microwave doors, with their part geometry and joints. Assets are scaled using known dimensions or depth, calibrated to the robot base, and converted to USD with joint and physical properties. Collision meshes preserve cavities and contact surfaces.

A queryable USD index [41] stores entity IDs, prim paths, poses, meshes, part hierarchies, and joint types, axes, and limits. Rendered masks link IDs to image regions; back-projected part masks and fitted interiors define contact patches and containment volumes for grounding.

B. Manipulation Knowledge Extraction and Reasoning

To distinguish task requirements from demonstration-specific choices, our knowledge extraction and reasoning module combines VLM-based parsing of object roles and subtask structure with recovered motion evidence. A pre-trained VLM with frozen parameters reasons over this information and target-scene descriptions, using commonsense knowledge to infer task conditions and permissible execution variations.

a) Video parsing: The VLM infers a task instruction from the video and identifies subtask boundaries from changes in manipulated objects, contacts, and object relations. Each subtask records its interval, object roles, initial and goal relations, contact/release events, and supporting frames. SAM3.1 [42] tracks object masks; SpatialTracker V2 [43] estimates depth and camera parameters; FoundationPose++ [44] uses these estimates and aligned SAM3D Objects meshes to track object and rigid-part poses. Motion relative to reference object b is ${}^bT_o(t) = ({}^CT_b(t))^{-1}CT_o(t)$, where C is the source camera frame and AT_B maps B coordinates to A . Without known source dimensions, translations are normalized by source extent and scaled to the corresponding target dimension. Motion intervals are linked to their subtasks.

b) Structured knowledge: We adopt CRAM’s object, location, and action designators [12]. Object designators describe properties and parts; location designators specify spatial relations and pose requirements; action designators reference these descriptions and contain execution parameters to be resolved in the target scene.

The VLM organizes the linked knowledge as $\mathcal{K} = (\mathcal{O}, \mathcal{S}, \mathcal{C}, \mathcal{R}, \mathcal{A})$: $\mathcal{O}$ contains object and part descriptions, $\mathcal{S}$ contains subtasks and their dependencies, $\mathcal{C}$ specifies required conditions, $\mathcal{R}$ stores contact and motion references, and $\mathcal{A}$ specifies permissible variations. Records link to the relevant designators and subtasks; target-scene resolution instantiates alternatives using available parts, geometry, and execution operators. Conditions specify predicates, role arguments, and whether they apply before, during, or after a subtask; variations identify candidate contact parts and



Fig. 2. Real-robot workspace with a Franka arm, parallel-jaw gripper, and fixed external camera.

supported interaction modes. Predicates cover containment, relative placement, orientation, joint state, release, and gripper clearance.

Observed order is recorded in $\mathcal{R}$, while precedence edges in $\mathcal{S}$ are inferred when one subtask produces a state required by another. Video observations carry evidence links and are distinguished from commonsense deductions. Structural checks verify role references, acyclic subtask dependencies, and supported predicates and modes. When ambiguities or inconsistencies arise, the VLM re-examines relevant clips and target-scene views; unresolved critical fields stop generation.

C. Knowledge-Guided Execution Generation in the Target Workspace

We use CRAM’s designator resolution mechanism to instantiate descriptions in the digital twin. Our resolution procedures combine VLM-based semantic matching, USD queries, and geometric sampling (Algorithm 1).

a) Entity and region grounding: The VLM uses the digital twin’s entity and part descriptions, spatial relations, joint states, geometric queries, and rendered views with ID overlays to match object references to scene entities. Each reference r is bound to an entity and region, $\beta(r) = (e_R, \rho_R)$. The scene index verifies entity existence, part membership, and joint compatibility. The location resolver converts spatial relations and pose requirements into sampling conditions; concrete poses are sampled after each layout update.

b) Execution selection: Given the inferred task conditions and permissible variations, the VLM uses digital-twin information to select candidate contact parts, interaction modes, and grasp strategies supported by the system. For grasping, it also selects a coarse approach direction compatible with subsequent placement, release, and withdrawal. Each alternative is $\xi = (p, m, h)$, with contact part/region p , interaction mode m , and grasp strategy h ; surface pushing uses $h = \emptyset$. For grasping alternatives, an associated approach-direction rule constrains geometric grasp sampling. Supported strategies include rim grasping, body grasping, and handle grasping, corresponding to contacts on the rim, outer body, and handle. The available parts determine which strategies apply to an object. The VLM retains multiple task-compatible alternatives.

A task-constraint adapter maps modes to grasp-and-transport, placement/release, grasped-articulation, or surface-pushing operators. Each operator provides geometric inputs, gripper states, preconditions/effects, and a waypoint constructor. Task dependencies and operator preconditions determine admissible subtask orders. Typed predicates check preconditions, propagated effects, contact regions, and goal conflicts before planning; continuous conditions remain attached to their subtasks for geometric and physical validation.

c) Geometric sampling and motion construction: The adapter obtains contact regions from bound parts and resolves admissible approach directions from their geometry. A grasp strategy may admit multiple approach directions.

We represent each admissible approach direction using the approach-cone formulation in CAPGrasp [45]. The adapter resolves a unit direction $\mathbf{d}$ in the robot-base frame from gravity, a part frame, or a surface normal. With angular tolerance α , a grasp g must satisfy

$$\mathbf{u}(g)^\top \mathbf{d} \geq \cos \alpha, \quad (1)$$

where $\mathbf{u}(g)$ is the unit pre-grasp-to-contact direction. The tolerance is specified by the geometric rule for that approach. Top-down and side describe approach directions; rim, body, and handle describe the grasp strategies whose contacts are constrained by the bound regions.

After each layout update, the Isaac Sim Grasping SDG antipodal sampler [46] generates concrete grasp poses from object meshes, with opposing surface contacts within the gripper aperture. The adapter retains poses satisfying the selected strategy’s grounded contact-region, approach-direction, and clearance constraints. Placement sampling accounts for destination boundaries, object footprint, support height, and orientation.

For compatible interaction modes, we express the recovered object motion relative to the task-relevant reference object or part. We map this motion into the corresponding target frame, scale translations to target dimensions, and adapt it to the sampled placement pose. For candidate a , its object waypoints and sampled grasp g_a determine the end-effector targets:

$${}^{W_R}T_e(t; a) = {}^{W_R}T_o(t; a) {}^oT_e(g_a), \quad (2)$$

where W_R is the robot-base frame and ${}^oT_e(g_a)$ is the grasp’s object-relative gripper pose. Alternative interaction modes construct waypoints from sampled contacts, goals, and operator geometry. Each candidate includes release and withdrawal.

d) Drawer example: In Fig. 1(b)–(c), the required outcome is a closed drawer with the banana inside. The human’s handle grasp is a reference; resolution yields handle-grasp and panel-push alternatives. Handle-based motion adapts the reference to the target joint geometry. Panel pushing uses the sampled contact, panel normal, joint axis, and drawer displacement. Both follow insertion and gripper withdrawal and satisfy the same containment and joint-state conditions (Fig. 3(a)).

Algorithm 1 Knowledge-guided demonstration generation

Require: Knowledge $\mathcal{K}$, indexed digital twin $\mathcal{E}_R$, attempt budget B

- 1: Resolve designators to bindings β , sampling conditions, and alternatives $\xi = (p, m, h)$ with grasp approach rules
 - 2: Enumerate admissible alternative/order combinations; initialize $D_R \leftarrow \emptyset$
 - 3: **for** $i = 1$ to B **do**
 - 4: Select the next combination in round-robin order; sample x_0 and update grounded geometry
 - 5: **if** symbolic conditions fail **then continue**
 - 6: Sample contacts, grasps, and goals under grounded constraints
 - 7: **if** geometric sampling fails **then continue**
 - 8: Construct waypoints from compatible references or operator geometry; plan the complete execution
 - 9: **if** planning fails **then continue**
 - 10: Reset to x_0 ; execute and monitor subtask conditions
 - 11: Verify execution labels against the rollout
 - 12: **if** all subtask conditions, final goals, and label checks pass **then**
 - 13: Store the task instruction, verified labels, observations, and actions in D_R
 - 14: **end if**
 - 15: **end for**
 - 16: **return** D_R
-

e) Planning: cuRobo [47] receives the joint state, collision world, carried-object geometry, and targets. Free-space segments use motion generation; interaction waypoints use sequential inverse kinematics initialized from the preceding solution. Interpolated paths are checked for joint limits, collisions, and orientation/region conditions, with intended contacts specified per subtask.

D. Simulation Validation and Policy Learning

Isaac Sim executes joint/gripper commands and checks subtask predicates using relative gripper-object motion, placement stability, joint states, and destination geometry. Acceptance requires all subtask conditions and final goals, including release and withdrawal; failures are logged by subtask. Accepted records synchronize RGB, proprioception, actions, knowledge, and bindings. They retain the shared task instruction and execution labels: target entities, contact parts, interaction modes, grasp strategies, and subtask order. Execution traces and contact checks verify labels initialized from candidate choices. An execution mode is defined by a contact strategy and a valid subtask order. Continuous contact positions and target-plate choices are not counted as separate modes. Labels support selection by execution mode and optional conditioning of policy training. Training actions comprise end-effector pose increments computed from consecutive executed poses and the corresponding binary gripper commands. Policy fine-tuning and real deployment use the same action representation. Section IV-D specifies the learning and deployment settings.

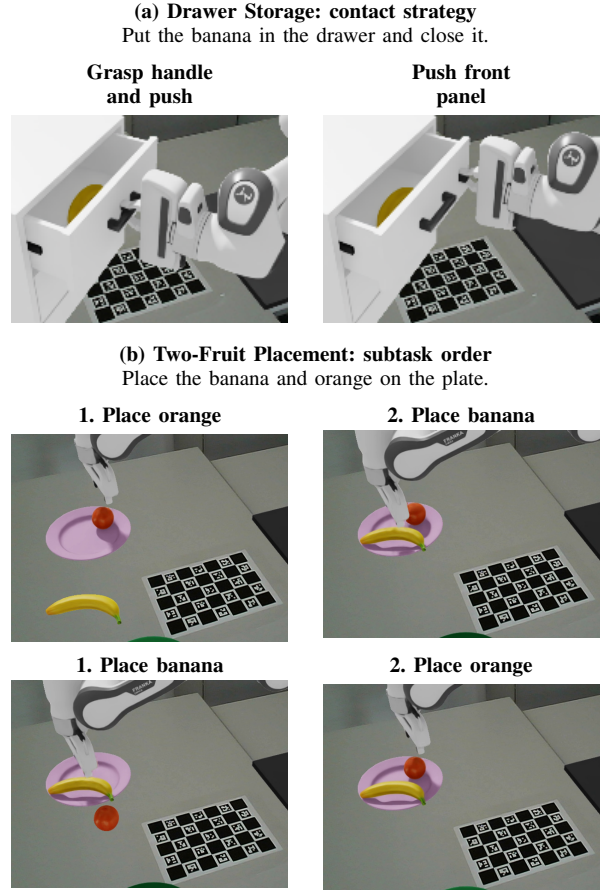


Fig. 3. Successful simulation executions under shared task instructions. For each task, alternatives use the same reference video and initial state. (a) Drawer closure by grasping the handle and pushing, or by pushing the front panel. (b) Both fruit-placement orders on the same plate, shown after each placement.

IV. EXPERIMENTS

We evaluate three aspects of the framework: behavioral diversity from the same human video, candidate planning success through task-guided grasp sampling, and the use of generated data for policy fine-tuning and sim-to-real transfer.

A. Experimental Setup

a) Platform and inputs: We use a Franka arm with a parallel-jaw gripper, a fixed external RGB camera, and the digital twin described in Section III-A (Fig. 2). We run the pretrained Qwen3.8-27B [48] with frozen parameters on a single NVIDIA A100 GPU. Videos are sampled at 2 FPS with timestamps; ambiguous contacts or subtask boundaries trigger VLM re-analysis of more densely sampled clips. Each task uses one reference video, recovered motion evidence, and target assets. Layouts vary object poses and include distractors.

b) Tasks: *Drawer Storage* opens a drawer, inserts a banana, withdraws the gripper from the interior, and closes the drawer with the banana inside; closure permits handle grasping or panel pushing. *Two-Fruit Placement* places an orange and a banana on a plate in either order, leaving both stable within their placement regions. *Close Microwave* requires full door closure and gripper withdrawal, allowing

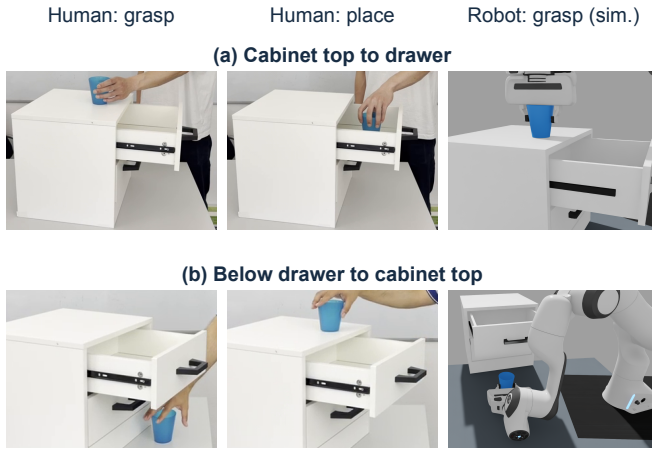


Fig. 4. Human grasp and placement references with corresponding robot grasps in simulation: (a) cabinet top to drawer, using a top-down robot approach; (b) beneath an extended drawer to cabinet top, pairing a palm-down human grasp with a side approach by the robot. Robot grasps accommodate placement and withdrawal in (a), and access beneath the drawer in (b).

TABLE I
VERIFIED EXECUTION MODES.

Task	Reference-only	KNOWDEMO
Drawer Storage	1	2
Two-Fruit Placement	1	2
Close Microwave*	1	1

*Close Microwave: both use one pushing mode; KNOWDEMO varies contact positions within that mode.

different pushing contacts. Real-policy evaluation uses these three tasks, with Banana Drawer Storage starting from an already open drawer.

Grasp-sampling evaluation uses the two tasks in Fig. 4: *Cup-in-Drawer* moves a cup from the cabinet top into an open drawer, and *Cup-to-Cabinet* moves a cup from beneath an extended drawer onto the cabinet top. Both require stability after release and gripper withdrawal. Success criteria are fixed independently of inferred knowledge.

c) *Generation backend*: The backend combines video-derived motion references, geometric grasp sampling, and planning, following the general workflow of Real2Gen [30], with task adapters for articulated motion and execution of multiple subtasks. The grasp-sampling comparison changes grasp-strategy selection within this shared backend, as detailed below.

B. Behavioral Diversity of Generated Demonstrations

Table I reports execution modes verified by complete successful demonstrations. *Reference-only* retains the demonstrated contact strategy and subtask order using the same backend for target-scene adaptation. Modes follow the definition in Section III-D.

Figure 3 shows successful alternatives from matched initial states. Drawer Storage uses handle grasping or panel pushing for closure after banana insertion and gripper withdrawal, preserving containment and the final closed state (Fig. 3(a)).

TABLE II
GRASP-SAMPLING ABLATION.

Task	Without task guidance	KNOWDEMO
Cup-in-Drawer	29/50	50/50
Cup-to-Cabinet	23/50	45/50

Entries report successful complete-task plans / sampled candidates.

For Two-Fruit Placement, KNOWDEMO varies placement order and, when task conditions permit, target plate; both are retained as execution labels. Figure 3(b) shows banana-first and orange-first sequences using the same plate, with frames after each placement.

For Close Microwave, the generated demonstrations use different door contact positions within the same pushing mode, while preserving the task instruction and requiring no additional human videos.

C. Task-Guided Grasp Sampling

Human grasp strategies need not transfer directly to a robot gripper. Figure 4 pairs grasp and placement frames from each reference video. In (a), the cup moves from the cabinet top into a drawer, where placement and gripper withdrawal constrain grasp choice. The human reorients the cup within the hand before insertion, so the grasp and placement frames show different hand-object configurations. In (b), the human uses a palm-down grasp to move the cup from beneath an extended drawer to the cabinet top; the overhang constrains robot approach and extraction. The framework preserves the task goals while selecting robot contact regions and approach directions from target geometry. Planning and simulation check the complete execution.

Task-oriented grasping connects contact selection to operation requirements [27]. Our quantitative ablation uses Cup-in-Drawer and Cup-to-Cabinet to measure whether task-guided strategy selection increases complete-task planning success under a fixed budget.

a) *Comparison*: We isolate task-guided grasp selection within the same generation backend. *Without task guidance*, a grasp strategy and its associated approach rule are sampled from the object’s applicable set independently of task-specific approach and placement requirements. KNOWDEMO selects strategies using task requirements and target-scene information. For the handleless cups, both configurations use the rim- and body-grasp strategies from Section III-C. The adapter maps each strategy to contact-region and approach-direction constraints. KNOWDEMO selects a top-down approach for Cup-in-Drawer to accommodate placement and withdrawal inside the drawer, and a side approach for Cup-to-Cabinet to accommodate access beneath the extended drawer. Both configurations use Grasping SDG with the same scene assets, grounded geometry, placement goals, execution operators, and per-candidate planning budget; task conditions and geometric checks remain active during planning and simulation validation. Initial positions vary within a small neighborhood of each setup, with paired states across meth-

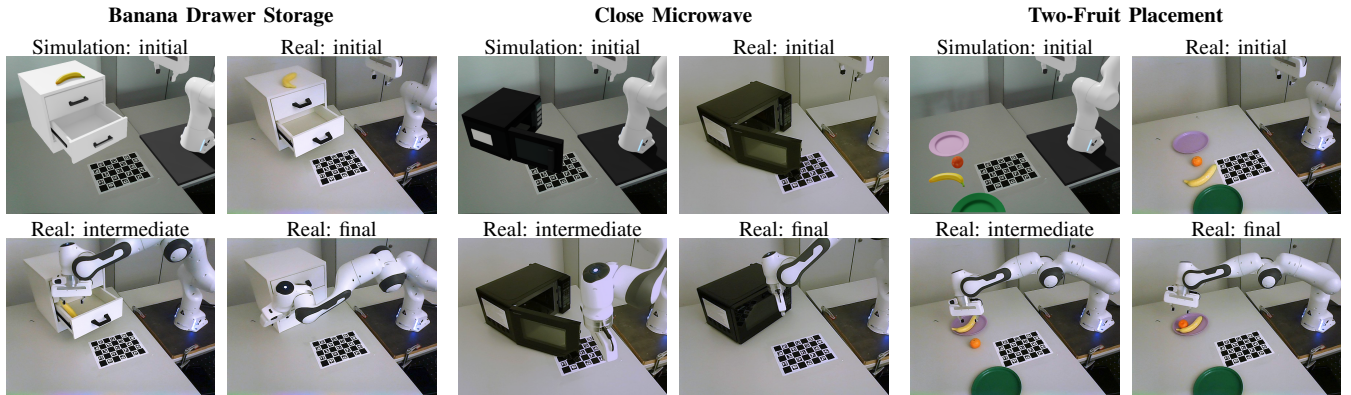


Fig. 5. Simulation observations and real-robot executions. For each task, the upper-left image combines rendered robots and task objects with a real background. The other three images show initial, intermediate, and final frames of autonomous real-robot executions using $\pi_{0.5}$ fine-tuned on generated simulation demonstrations.

TABLE III
REAL-ROBOT POLICY SUCCESS RATES.

Task	Simulation demos	Successes/trials	Success rate (%)
Banana Drawer Storage	200	10/20	50
Close Microwave	100	14/20	70
Two-Fruit Placement	200	11/20	55

ods. Each method submits 50 candidates per task; a plan must cover grasp, transfer, placement, and withdrawal.

b) Results: Table II reports complete-task plans out of 50 candidates per task and method, evaluated before simulation execution. Task-guided grasp selection improves planning success from 58% to 100% for Cup-in-Drawer and from 46% to 90% for Cup-to-Cabinet. Failed candidates include grasps that permit pickup but prevent the planner from finding a collision-free trajectory for subsequent placement or gripper withdrawal. Task-guided selection accounts for these downstream requirements when choosing the grasp strategy.

D. Policy Learning and Real-Robot Evaluation

We fine-tune the pretrained $\pi_{0.5}$ vision-language-action model [19] for each task evaluated on the real robot, using only generated simulation demonstrations. Policies receive RGB images, proprioceptive states, and the shared task instructions. Execution labels are retained as dataset metadata. We perform full-parameter fine-tuning with openpi for 15,000 steps, using a batch size of 256 and a learning rate of 5×10^{-5} . The action chunk length is 10, with all 10 actions executed before the next query.

Table III reports dataset sizes and complete-task success over 20 real trials per task with layout variations and no human intervention. Success rates of 50–70% demonstrate sim-to-real transfer for door pushing, sequential fruit placement, and object placement followed by drawer closure. Figure 5 pairs simulation initial states with autonomous real executions. The learned policies also exhibit behavioral diversity: the same Two-Fruit Placement policy successfully uses both placement orders, and Close Microwave executions use different door contact positions.

Failures occur when target objects are occluded or when the microwave and drawer cabinet positions differ substantially from those represented in the generated demonstrations. These observations indicate sensitivity to occlusion and spatial layout changes in the deployed policies.

V. LIMITATIONS

The framework depends on the quality of inferred knowledge and the target scene model. Task ambiguity, occlusion, and motion-recovery errors can lead to incorrect task conditions, excluding valid alternatives or admitting inappropriate ones. Geometric and physical validation checks execution against the inferred conditions but cannot establish whether those conditions capture the intended task. Errors in target geometry, articulation, and contact parameters, together with visual and dynamic differences between simulation and hardware, can degrade generated demonstrations and real-world execution. Extending the framework to new tasks requires compatible assets, support for the required interactions, and policy fine-tuning. Multiple demonstrations and real execution feedback could improve condition inference and scene-model accuracy.

VI. CONCLUSION

We presented KNOWDEMO, a framework that uses structured manipulation knowledge from human videos to guide robot demonstration generation in a target workspace. To expand behavioral diversity while preserving task requirements, our VLM-based knowledge extraction and reasoning module distinguishes required conditions, demonstration references, and permissible variations within CRAM action descriptions. To reduce invalid generation attempts, we ground these descriptions in target-scene entities and geometry to guide candidate sampling and screening before planning and simulation. The resulting demonstrations share the inferred task instruction and retain structured execution labels. Experiments demonstrate additional verified execution modes beyond a reference-only configuration, improved candidate planning success through task-guided grasp sampling, and sim-to-real transfer using $\pi_{0.5}$ fine-tuned on generated demonstrations.

REFERENCES

- [1] Open X-Embodiment Collaboration, “Open X-Embodiment: Robotic learning datasets and RT-X models,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 6892–6903.
- [2] A. Khazatsky, *et al.*, “DROID: A large-scale in-the-wild robot manipulation dataset,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2024.
- [3] A. Mandlekar, *et al.*, “MimicGen: A data generation system for scalable robot learning using human demonstrations,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 229, 2023, pp. 1820–1864.
- [4] Z. Xue, S. Deng, Z. Chen, Y. Wang, Z. Yuan, and H. Xu, “DemoGen: Synthetic demonstration generation for data-efficient visuomotor policy learning,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2025.
- [5] V. Jain, *et al.*, “Vid2Robot: End-to-end video-conditioned policy learning with cross-attention transformers,” 2024, arXiv:2403.12943.
- [6] J. Shi, *et al.*, “ZeroMimic: Distilling robotic manipulation skills from web videos,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025. [Online]. Available: <https://zeromimic.github.io/>
- [7] P. Dan, *et al.*, “X-Sim: Cross-embodiment learning via real-to-sim-to-real,” 2025, arXiv:2505.07096.
- [8] B. Wang, J. Zhang, S. Dong, I. Fang, and C. Feng, “VLM see, robot do: Human demo video to robot action plan via vision language model,” 2024, arXiv:2410.08792.
- [9] N. Heppert, M. Argus, T. Welschhold, T. Brox, and A. Valada, “DITTO: Demonstration imitation by trajectory transformation,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2024, pp. 7565–7572.
- [10] J. Mu, *et al.*, “DexImit: Learning bimanual dexterous manipulation from monocular human videos,” 2026, arXiv:2602.10105.
- [11] K. Lin, *et al.*, “Constraint-preserving data generation for one-shot visuomotor policy generalization,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 305, 2025, pp. 3631–3646.
- [12] M. Beetz, G. Kazhoyan, and D. Vernon, “Robot manipulation in everyday activities with the Cram 2.0 cognitive architecture and generalized action plans,” *Cogn. Syst. Res.*, vol. 92, p. 101375, 2025.
- [13] C. R. Garrett, A. Mandlekar, B. Wen, and D. Fox, “SkillMimicGen: Automated demonstration generation for efficient skill learning and deployment,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 270, 2025, pp. 2750–2790.
- [14] W. Wang and N. Tan, “HybridGen: VLM-guided hybrid planning for scalable data generation of imitation learning,” 2025, arXiv:2503.13171.
- [15] A. J. Zhai, K.-H. Zeng, J. Lu, A. Farhadi, S. Wang, and W.-C. Ma, “Imitating what works: Simulation-filtered modular policy learning from human videos,” *Trans. Mach. Learn. Res.*, 2026. [Online]. Available: <https://openreview.net/forum?id=ZEmv4DhaGL>
- [16] C. Chi, *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2023.
- [17] N. M. M. Shafiuallah, Z. J. Cui, A. Altanzaya, and L. Pinto, “Behavior transformers: Cloning k modes with one stone,” in *Adv. Neural Inf. Process. Syst.*, vol. 35, 2022.
- [18] X. Jia, *et al.*, “Towards diverse behaviors: A benchmark for imitation learning with human demonstrations,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
- [19] Physical Intelligence, *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” 2025, arXiv:2504.16054.
- [20] C. Wang, *et al.*, “MimicPlay: Long-horizon imitation learning by watching human play,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 229, 2023, pp. 201–221.
- [21] Y. Zhu, A. Lim, P. Stone, and Y. Zhu, “Vision-based manipulation from single human video with open-world object graphs,” *Auton. Robots*, vol. 50, 2026, Art. no. 27.
- [22] S. Park, H. Bharadhwaj, and S. Tulsiani, “DemoDiffusion: One-shot human imitation using pre-trained diffusion policy,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2026.
- [23] M. Beetz, D. Beßler, A. Haidu, M. Pomarlan, A. K. Bozcuoğlu, and G. Bartels, “KnowRob 2.0—a 2nd generation knowledge processing framework for cognition-enabled robotic agents,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2018, pp. 512–519.
- [24] L. Manuelli, W. Gao, P. Florence, and R. Tedrake, “kPAM: Key-Point affordances for category-level robotic manipulation,” 2019, arXiv:1903.06684.
- [25] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei, “ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 270, 2025, pp. 4573–4602.
- [26] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: Composable 3D value maps for robotic manipulation with language models,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 229, 2023, pp. 540–562.
- [27] E. Tong, A. Balaji, A. Opipari, S. Lewis, Z. Zeng, and O. C. Jenkins, “OVAL-Grasp: Open-vocabulary affordance localization for task oriented grasping,” in *Proc. Int. Symp. Exp. Robot. (ISER)*, 2025. [Online]. Available: <https://ekjt.github.io/OVAL-Grasp/>
- [28] C. Mao, H. Yuan, Z. Huang, C. Xu, K. Ma, and Z. Lu, “DemoFunGrasp: Universal dexterous functional grasping via demonstration-editing reinforcement learning,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026, pp. 986–995.
- [29] J. Zhang, K. Hu, Y. Huang, Y. Ju, Z. Xue, and H. Xu, “AffordGen: Generating diverse demonstrations for generalizable object manipulation with affordance correspondence,” 2026, arXiv:2604.10579.
- [30] N. Heppert, M. Q. Nguyen, and A. Valada, “Scaling single human demonstrations for imitation learning using generative foundational models,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2026.
- [31] J. Luo, “From passive video to editable experience: Physically grounded experience synthesis for embodied intelligence,” 2026, arXiv:2607.26903.
- [32] W. Ye, F. Liu, Z. Ding, Y. Gao, O. Rybkin, and P. Abbeel, “Video2Policy: Scaling up manipulation tasks in simulation through internet videos,” 2025, arXiv:2502.09886.
- [33] T. G. W. Lum, O. Y. Lee, C. K. Liu, and J. Bohg, “Crossing the human-robot embodiment gap with sim-to-real RL using one human demonstration,” 2025, arXiv:2504.12609.
- [34] Y. Weng, *et al.*, “Neural implicit representation for building digital twins of unknown articulated objects,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 3141–3150.
- [35] X. Li, *et al.*, “Evaluating real-world robot manipulation policies in simulation,” 2024, arXiv:2405.05941.
- [36] M. N. Qureshi, S. Garg, F. Yandun, D. Held, G. Kantor, and A. Silwal, “SplatSim: Zero-shot Sim2Real transfer of RGB manipulation policies using gaussian splatting,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025, pp. 6502–6509.
- [37] H. Zhao, *et al.*, “High-fidelity simulated data generation for real-world zero-shot robotic manipulation learning with gaussian splatting,” 2025, arXiv:2510.10637.
- [38] N. Carion, *et al.*, “SAM 3: Segment anything with concepts,” 2025, arXiv:2511.16719.
- [39] SAM 3D Team, *et al.*, “SAM 3D: 3Dfy anything in images,” 2025, arXiv:2511.16624.
- [40] Z. Wu, *et al.*, “URDF-Anything+: End-to-end generation for simulation-ready articulated assets,” 2026, arXiv:2603.14010.
- [41] G. H. Nguyen, D. Beßler, S. Stelter, M. Pomarlan, and M. Beetz, “Translating universal scene descriptions into knowledge graphs for robotic environment,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 9389–9395.
- [42] SAM 3 Team, “SAM 3.1: Release notes,” GitHub, 2026, Accessed: Sep. 14, 2026. [Online]. Available: https://github.com/facebookresearch/sam3/blob/main/RELEASE_SAM3p1.md
- [43] Y. Xiao, *et al.*, “SpatialTrackerV2: Advancing 3D point tracking with explicit camera motion,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2025, pp. 6726–6737.
- [44] teal024, “FoundationPose++: Simple tricks boost FoundationPose performance in high-dynamic scenes,” GitHub, 2025, Accessed: Sep. 14, 2026. [Online]. Available: <https://github.com/teal024/FoundationPose-plus-plus>
- [45] Z. Weng, H. Lu, J. Lundell, and D. Kragic, “CAPGrasp: An $\mathbb{R}^3 \times \text{SO}(2)$ -equivariant continuous approach-constrained generative grasp sampler,” *IEEE Robot. Autom. Lett.*, vol. 9, no. 4, pp. 3641–3647, 2024.
- [46] NVIDIA, “Grasping synthetic data generation,” Isaac Sim 5.1 Documentation, accessed: Sep. 15, 2026. [Online]. Available: https://docs.isaacsim.omniverse.nvidia.com/5.1.0/synthetic_data_generation/tutorial_replicator_grasping_sdg.html
- [47] B. Sundaralingam, *et al.*, “CuRobo: Parallelized collision-free robot motion generation,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 8112–8119.

[48] Qwen Team, “Qwen3.8-27B,” Hugging Face, 2026, model card. Accessed: Sep. 14, 2026. [Online]. Available: <https://huggingface.co/Qwen/Qwen3.8-27B>